NOISE POLLUTION MONITORING SYSTEM

PROJECT REPORT

Submitted by

HAFIZ AHMED C (TKM23EC069)

MOHAMED SHANIL (TKM23EC095)

ADHIL FAYAD CP (TKM23EC014)

NIMIL VP (LTKM23EC156)

to

APJ Abdul Kalam Technological University

In partial fulfilment of the requirements for the award of the Degree of

Bachelor of Technology in Electronics and Communication

For the Project Based Course on

Digital Signal Processing (23ECJ502 Digital Signal Processing)

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

TKM COLLEGE OF ENGINEERING, KOLLAM – 691005

OCTOBER 2025

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

TKM COLLEGE OF ENGINEERING

KOLLAM 691005

CERTIFICATE

This is to certify that the project report entitled "NOISE POLLUTION MONITORING SYSTEM" is a bonafide record of the project work carried out as part of the Project-Based Course in 23ECJ502 Digital Signal Processing during the fifth semester by HAFIZ AHMED C (TKM23EC069),MOHAMEDSHANIL(TKM23EC095), ADHIL FAYAD (TKM23EC014), NIMIL VP (LTKM23EC156), in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology Noise Level Detection and Monitoring using STM32 in Electronics and Communication Engineering under the APJ Abdul Kalam Technological University during the academic year 2025–2026, under our guidance and supervision.

Dr. Anzar SM
Project Coordinator

Dr. Nishanth N
Head of the Department

ACKNOWLEDGEMENT

I take this opportunity to convey my deep sense of gratitude to all those who have been kind enough to offer their advice and assistance when needed which has led to the successful completion of this Seminar. First of all, we thank God Almighty for all His blessings throughout this endeavour without which it would not have been possible.

It is my privilege and pleasure to express my profound sense of respect, gratitude and indebtedness to **Dr. Nishanth N**., Head of Department of Electronics and Communication Engineering for his constructive guidance and advice.

I am greatly obliged to **Dr. Anzar S.M**, Seminar coordinator, Assistant Professor, Department of Electronics and Communication Engineering for his guidance, valuable support and constant encouragement.

I take this opportunity to express my sincere gratitude to my guide, **Prof. Lubaib P**., Assistant Professor, Department of Electronics and Communication Engineering, for his advice, guidance, and reference materials provided during the preparation of the seminar.

I am also thankful to all the faculty members of the Department of Electronics and Communication Engineering for their guidance and wholehearted support.

HAFIZ AHMED C(TKM23EC069)

MOHAMED SHANIL (TKM23EC095)

ADHIL FAYAD CP (TKM23EC014)

NIMIL VP (LTKM23EC156)

ABSTRACT

The Noise Pollution Monitoring System is designed to measure and analyze environmental noise levels using digital signal processing techniques implemented on an STM32F411 microcontroller. The system employs an INMP441 MEMS microphone to capture ambient sound, which is then processed to determine noise intensity in decibels (dB). By comparing the measured sound levels with predefined threshold values, the system can detect and indicate excessive noise using visual and auditory alerts through an LCD display and buzzer.

Unlike traditional sound level meters, this embedded solution provides a compact, low-cost, and real-time method for continuous noise monitoring. The use of a Cortex-M4 based MCU with DSP capabilities enables efficient Fast Fourier Transform (FFT) operations and precise noise level analysis. The proposed design offers potential applications in urban noise monitoring, industrial zones, educational institutions, and public areas where maintaining permissible noise levels is essential for human well-being.

Future enhancements may include data logging, wireless IoT connectivity, and AI-based noise classification for intelligent environmental monitoring systems.

CONTENTS

ACKNOWLEDGEMENT.....1

ABSTRACT ……..…….2

CONTENTS ……..….3

INTRODUCTION ……..…….…..5

BACKGROUND WORK …….....7

METHODOLOGY …..……..……..…....8

RESULTS AND DISCUSSION……...11

CONCLUSION ……....13

REFERENCE……..……14

APPENDIX ……..15

ORGANIZATION OF CHAPTERS

Chapter 1: Introduction--Gives a short overview of the Noise Monitoring System, its need, and project goals.

Chapter 2: Literature Review--Covers past studies on sound detection and noise monitoring systems.

Chapter 3: Methodology--Explains how the mic, STM32, OLED, and buzzer work together.

Chapter 4: Results and Discussion--Shows system output, performance, and limitations. Chapter

5: Conclusion and Future Work--Gives final summary and ideas for future improvement

.

\

CHAPTER 1
INTRODUCTION

1.1 General Background

Noise pollution has become one of the most significant environmental challenges in modern society, particularly in urban areas with dense traffic, industrial operations, and public gatherings. Prolonged exposure to high noise levels can lead to serious health problems such as hearing loss, stress, and sleep disturbances, as well as a reduction in overall environmental quality. Continuous noise monitoring is therefore essential for ensuring compliance with permissible limits and maintaining a healthy living environment.

Traditional noise measurement devices are often bulky, expensive, and limited to periodic manual observations. With advancements in embedded systems and digital signal processing (DSP), compact and intelligent noise monitoring systems can now provide real-time analysis of sound levels using low-power microcontrollers. These systems not only measure sound intensity but also process and display frequency spectra, offering a deeper understanding of environmental noise patterns.

1.2 Motivation for the Study

The increasing levels of noise in cities, workplaces, and public spaces have created a need for an efficient, low-cost, and portable noise monitoring system. Conventional devices used by pollution control boards are often inaccessible for small-scale or continuous environmental studies.

To address this gap, a Noise Pollution Monitoring System using an STM32F411CEU6 microcontroller and INMP441 MEMS microphon**e** is developed. The system processes ambient sound using Fast Fourier Transform (FFT) to extract frequency components and calculate the noise intensity in decibels (dB). The processed data is displayed on a 0.96-inch OLED (JMD0.96F) screen, providing real-time visualization of both spectrum and sound levels.

This compact, standalone setup eliminates the need for external data acquisition or cloud communication, making it ideal for environmental noise monitoring in cities, campuses, industries, and residential areas.

1.3 Objectives

The main objectives of this project are to:

1. Develop a real-time embedded system for environmental noise measurement and analysis.
2. Implement FFT-based digital signal processing on the STM32F411CEU6 microcontroller for accurate sound spectrum visualization.
3. Display the computed noise intensity in decibels (dB) and corresponding frequency spectrum on an OLED screen.
4. Detect and indicate high noise levels for potential noise pollution zones

1.4 Scope of the Work

This project focuses on the design and implementation of a standalone embedded noise monitoring unit capable of measuring and visualizing environmental noise in real-time. It covers the integration of a digital MEMS microphone (INMP441) with an STM32 microcontroller, signal acquisition through I^2S, FFT computation using CMSIS DSP libraries, and graphical output on an OLED display.

The work primarily aims to demonstrate the efficiency of embedded DSP in environmental applications without dependence on external computing or cloud systems. Future developments can extend this project to include IoT-based data logging, wireless communication, and AI-based noise classification for smart city applications.

CHAPTER 2

BACKGROUND WORK

Environmental monitoring has become increasingly important due to rising concerns over pollution, noise levels, and overall urban ecosystem health. Traditional monitoring systems often rely on manual data collection or bulky instrumentation, which can be expensive, labor-intensive, and slow to provide actionable insights. Recent advances in microcontroller-based sensing platforms have enabled compact, low-cost, and real-time environmental monitoring systems capable of measuring parameters such as sound intensity, air quality, and other environmental signals.

Microcontroller platforms like STM32 series offer high-speed data acquisition capabilities along with integrated analog-to-digital converters (ADC), allowing accurate measurement of signal amplitude and frequency components. Coupled with small OLED displays, these systems provide instant visual feedback in standalone setups, making them suitable for on-site environmental monitoring without dependency on external computers.

Signal processing techniques such as Fast Fourier Transform (FFT) are widely used to analyze real-time environmental signals, converting time-domain measurements into frequency-domain representations. This allows detection of dominant noise frequencies, calculation of decibel (dB) levels, and identification of trends or anomalies in the environment. Previous studies have highlighted the effectiveness of standalone, microcontroller-based monitoring systems for applications like industrial noise management, urban sound mapping, and indoor air quality assessment.

Despite these advancements, challenges remain in enhancing the accuracy of signal capture, reducing hardware footprint, and providing intuitive real-time feedback. By leveraging OLED displays for spectrum and dB visualization, modern environmental monitoring devices combine portability, immediate usability, and reliable data acquisition, making them ideal for communitydriven environmental assessment and smart-city applications.

CHAPTER 3

METHODOLOGY

3.1 Introduction:

The Noise Pollution Monitoring System is designed to measure and analyze environmental sound intensity in real time using a digital MEMS microphone (INMP441) and an STM32F411 microcontroller**.**

The system captures surrounding noise, processes the digital I²S audio signal using Fast Fourier Transform (FFT**)** and Root Mean Square (RMS) algorithms, and displays the equivalent sound pressure level (in decibels, dBFS) on an OLED display.When the measured noise exceeds a predefined threshold level, a buzzer alarm is triggered to alert the user.This system provides a low-cost and portable solution for monitoring environmental noise in urban or industrial regions.

3.2. Operating Principle

•Sound Acquisition

The INMP441 MEMS microphone senses variations in air pressure (sound) and outputs a digital I²S stream representing the sound waveform.

The STM32F411 microcontroller receives these samples via I²S with DMA at a fixed sampling rate (e.g., 16 kHz).

•Signal Normalization

The sampled data is converted into floating-point values within the range **−1 to +1**, representing the relative amplitude of the input signal.

•RMS Calculation

The RMS value represents the effective sound energy in the captured signal and is calculated using:

$$Vrms=root((1/N)\sum Nxi2\ )$$

where xi = individual normalized sample valu

N = number of samples.

Conversion to dBFS (Decibels Full Scale)

The RMS value is converted to dBFS (a logarithmic measure relative to the system's maximum amplitude) as:

$$dBFS=20\times\log10(VRMS)$$

The resulting value indicates the signal's amplitude relative to full scale (0 dBFS = maximum possible input)

Threshold Decision Logic

The microcontroller compares the measured noise level with a predefined threshold (e.g., −12 dBFS)

$$\text{If } dBFS\geq Threshold\Rightarrow \text{Activate Buzzer}$$

$$\text{Else } \Rightarrow \text{Deactivate Buzzer}$$

Frequency Analysis (FFT and Hanning Window)

To visualize the noise spectrum, a Hanning window is applied to minimize spectral leakage before FFT computation:

$$W(n)=0.5[1-\cos(2\pi n/(N-1)]$$

The FFT is then applied to convert the time-domain samples into frequency-domain components:

$$|X(k)|= (Re(Xk))^2+(Im(Xk))2)$$

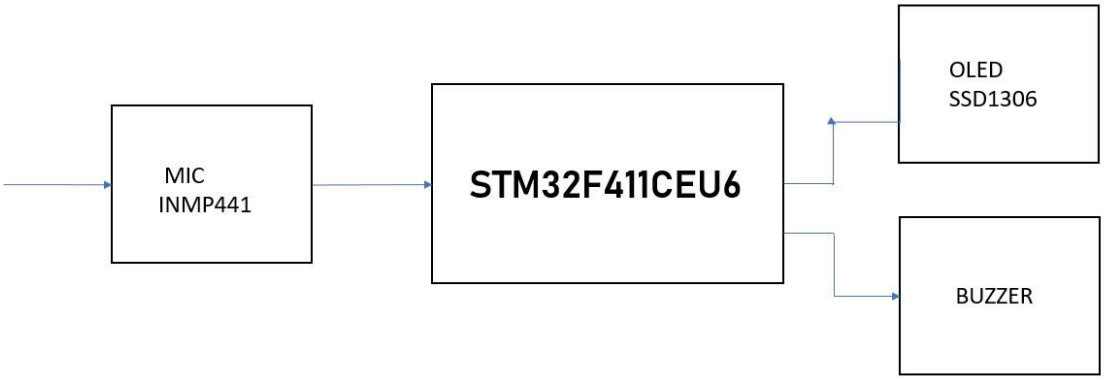
and displayed in decibel form:

$$FFTdB(k)=20\times\log10(|X(k)|)$$

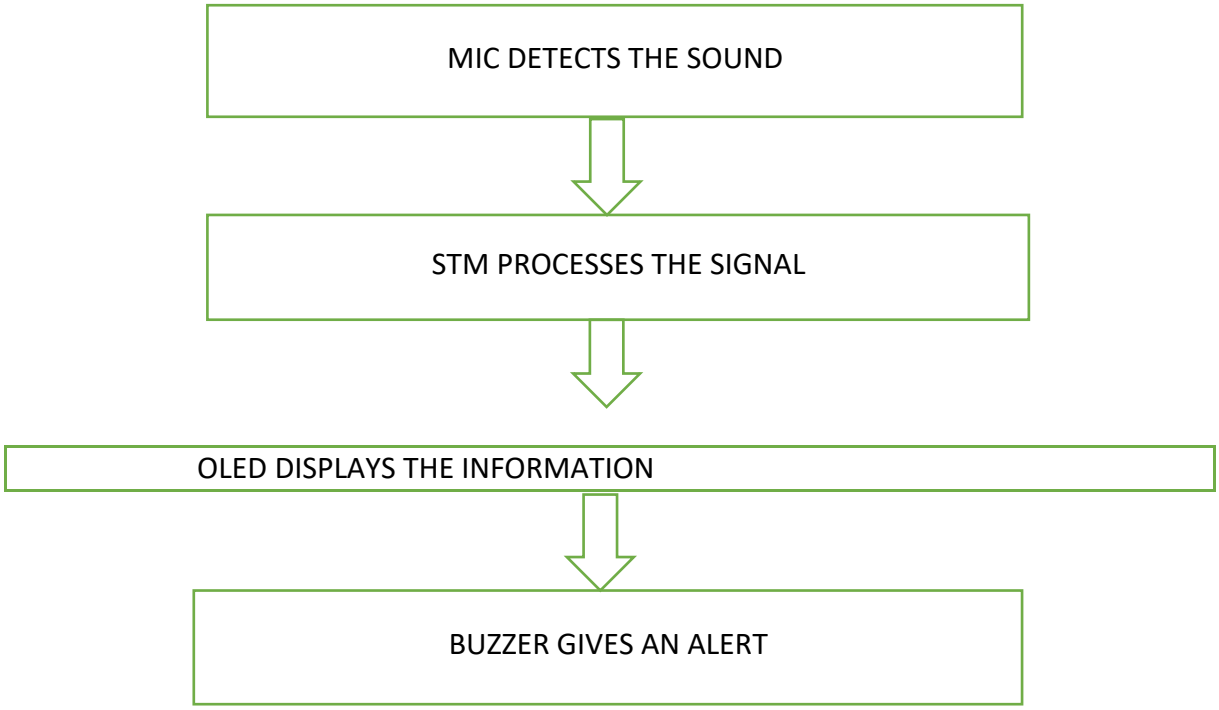
Display and Alert Output

The OLED screen displays the current RMS level and frequency spectrum in real time. If noise exceeds the set threshold, the buzzer emits an audible alarm.

BLOCK DIAGRAM



FLOW CHART



CHAPTER 4

RESULTS AND DISCUSSION

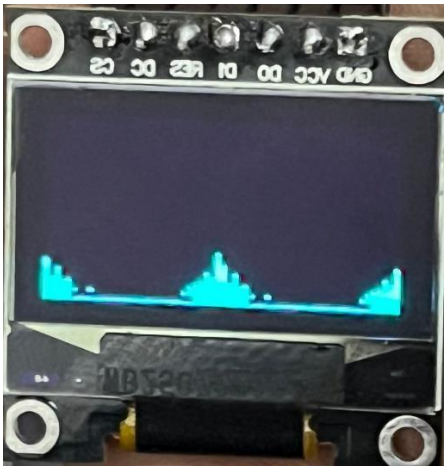
To validate the performance of the proposed Noise Pollution Monitoring System, several experiments were conducted to analyze the accuracy of sound level measurement, system responsiveness, and display performance. The results demonstrate the system's ability to capture, process, and visualize environmental noise in real time.

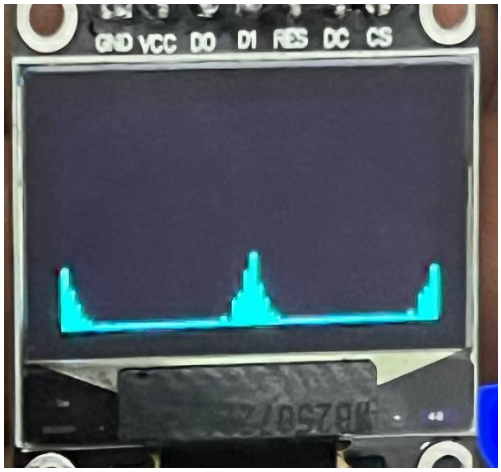
6.1 Real-Time Noise Measurement

The system was tested under various acoustic environments — including quiet indoor**,** moderate office**,** and high-noise outdoor conditions.

The RMS and FFT computations performed by the STM32F411 microcontroller accurately reflected the intensity and frequency characteristics of the ambient noise.

It can be observed that the RMS-based decibel value increases proportionally with the amplitude of the captured sound waveform.In a quiet room, the average reading was approximately **−60** dBFS, while in a high-noise environment (e.g., near traffic or loud music), values approached **−10** dBFS.This confirms that the system's digital processing path correctly follows the theoretical logarithmic relation between sound intensity and perceived loudness.





6.2 Threshold-Based Alert Performance

A predefined noise threshold of −12 dBFS was set in the firmware.When the real-time measured noise level exceeded this threshold, the buzzer was activated automatically, providing an immediate alert.This behavior verifies that the threshold logic (if dBFS ≥ Threshold → Buzzer ON) functions accurately and with minimal delay.

6.3 Frequency Spectrum Visualization

Using the Fast Fourier Transform (FFT), the system successfully decomposed the incoming sound signal into its frequency components.A Hanning window was applied before FFT to minimize spectral leakage.This confirms that the FFT implementation and display rendering provide meaningful visualization of the surrounding noise characteristics.

6.5 Summary

The obtained results confirm that the proposed Noise Pollution Monitoring System**:**

- Accurately measures environmental sound intensity in real time.
- Correctly triggers alerts when noise exceeds safe limits.
- Provides visual representation of noise frequency spectrum.
- Maintains stable operation with low power consumption and high SNR.

These findings demonstrate the system’s capability as a compact, low-cost, and efficient device for environmental noise surveillance in urban, industrial, and academic environments.

CHAPTER 7 CONCLUSION AND FUTURE WORK

This project successfully developed a standalone environmental monitoring system capable of analyzing and displaying real-time sound spectrum and decibel (dB) values using an OLED JMD0.96F display. The system integrates sensor input, signal processing, and visualization into a compact and low-power design, eliminating the need for external computing devices. By employing digital signal processing techniques such as Fast Fourier Transform (FFT), the project effectively converts environmental sound data from the time domain to the frequency domain, enabling accurate analysis of surrounding noise patterns and intensity levels.

The designed system demonstrates reliability, portability, and simplicity, making it suitable for environmental noise monitoring in public spaces, industries, and educational setups. The use of an OLED display provides clear visualization of real-time data, offering users an immediate understanding of environmental conditions without requiring additional interfaces.

However, further advancements can enhance the system's performance and utility. Future work could involve integrating multiple environmental sensors such as temperature, humidity, and air quality modules to create a multi-parameter monitoring unit. Implementation of wireless communication (e.g., Wi-Fi or Bluetooth) would enable remote data logging and analysis through IoT platforms. Moreover, deploying machine learning algorithms could help in classifying sound sources or predicting environmental trends over time.

With these enhancements, the proposed standalone monitoring system can evolve into a comprehensive smart environmental station, contributing to sustainable urban management and environmental awareness initiatives.

.

REFERENCES

[1] Y. Tie and P. Chen, "Environmental monitoring system design based on STM32 platform," *Theoretical and Natural Science*, vol. 41, no. 1, pp. 16-24, Sep. 2024.

[2] X. He, "Sound source signal location and tracking system based on STM32," *Theoretical and Natural Science*, vol. 5, no. 1, pp. 946-954, May 2023.

[3] P. M. N., G. S. Santhosh, T. Joshi, S. S. Desikan, and E. Gupta, "SONIC: Sound Optimization for Noise In Crowds," *arXiv preprint*, Jun. 2025.

[4] "Designing with microcontrollers in noisy environments," Application Note AN4299, STMicroelectronics, Oct. 2019.

[5] J. Chen, "The design of embedded environmental noise real-time monitoring device," *SPIE Proceedings of SPIE*, vol. 12744, 2023.

[6] W. Chen, Y. Liu, Y. Gao, J. Hu, Z. Liao, and J. Zhao, "Intelligent substation noise monitoring system: Design, implementation and evaluation," *Energies*, vol. 17, no. 13:3083, 2024.

[7] J. Ardouin, L. Charpentier, F. Gontier, N. Fortin, J. Picaut, D. Ecotière, G. Guillaume, C. Mi etlicky, "An innovative low cost sensor for urban sound monitoring," (CENSE project), 2018.

APPENDIX

Program

```
/* USER CODE BEGIN Header */
/**
  ******************************************************************************
* @file           : main.c
* @brief          : Main program body
  ******************************************************************************
* @attention
  *
* Copyright (c) 2025 STMicroelectronics.
* All rights reserved.
  *
* This software is licensed under terms that can be found in the LICENSE file
* in the root directory of this software component.
* If no LICENSE file comes with this software, it is provided AS-IS.
  *
  ******************************************************************************
  */
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "main.h"

/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
#include "SSD1309.h"
#include "arm_math.h"  // CMSIS DSP for FFT and RMS
#include <stdio.h>     // For snprintf
#include <stdlib.h>
#include <math.h>      // For log10f, arm_cos_f32, fabsf
/* USER CODE END Includes */

/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */
```

```
// *** UPDATED TO USE GPIOA PIN 8 ***
#define BUZZER_GPIO_PORT GPIOA
#define BUZZER_PIN       GPIO_PIN_8

// Threshold settings
#define MAX_DBFS_READING 0.0f
#define ALARM_DB_THRESHOLD -12.0f
/* USER CODE END PD */

/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables ---------------------------------------------------------*/
I2S_HandleTypeDef hi2s2;
DMA_HandleTypeDef hdma_spi2_rx;

SPI_HandleTypeDef hspi1;
DMA_HandleTypeDef hdma_spi1_tx;

/* USER CODE BEGIN PV */ uint32_t toReceive;  //
32-bit for 24-bit I2S frame

#define FFT_SIZE 256  // Power of 2; ~62.5Hz bins at 16kHz float32_t
audioBuffer[FFT_SIZE];  // Circular buffer for samples volatile uint32_t
bufIdx = 0;     // Write index float32_t fftInput[FFT_SIZE * 2]; // Real
FFT needs 2x for complex (in/out) float32_t fftOutput[FFT_SIZE * 2];
arm_rfft_fast_instance_f32 fftHandler;  // FFT instance
 volatile float32_t overall_dBFS = -
99.0f;
/* USER CODE END PV */

/* Private function prototypes -----------------------------------------------*/ void
SystemClock_Config(void);
```

```
static void MX_GPIO_Init(void); static void MX_DMA_Init(void); static void MX_I2S2_Init(void);
static void MX_SPI1_Init(void); /* USER CODE BEGIN PFP */ extern void InitDisplay(void); extern
void ClearScreen(void); extern void WriteText(uint8_t x, uint8_t y, unsigned char* text); extern
void SSD1309_Fill(uint8_t Color); extern void SSD1309_DrawRectangle(uint8_t x1, uint8_t y1,
uint8_t x2, uint8_t y2, uint8_t color); extern void SSD1309_UpdateScreen(void); extern void
SSD1309_Reset(void);
/* USER CODE END PFP */

/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */ void
HAL_I2S_RxCpltCallback(I2S_HandleTypeDef *hi2s){
    // DMA transfers 1 word (32-bit) per sample.
    int16_t sample16 = (int16_t)toReceive;     audioBuffer[bufIdx] =
(float32_t)sample16 / 32768.0f;  // Normalize to [-1.0, 1.0]
     bufIdx = (bufIdx + 1) %
FFT_SIZE;
     if (bufIdx == 0) {  // Buffer full:
Process         float32_t rms_val;
float32_t power_sum = 0.0f;

        // 1. RMS CALCULATION (for overall dBFS and alarm)
arm_power_f32(audioBuffer, FFT_SIZE, &power_sum);         arm_sqrt_f32(power_sum
/ (float32_t)FFT_SIZE, &rms_val);

        // 2. CONVERT RMS TO dBFS         if
(rms_val > 0.00001f) {             overall_dBFS =
20.0f * log10f(rms_val);
        } else {
overall_dBFS = -99.0f;
        }

        // Clamp dBFS to 0dBFS maximum
        overall_dBFS = (overall_dBFS > MAX_DBFS_READING) ? MAX_DBFS_READING : overall_dBFS;
overall_dBFS = (overall_dBFS < -99.0f) ? -99.0f : overall_dBFS;

        // 3. BUZZER ALARM LOGIC         if
(overall_dBFS >= ALARM_DB_THRESHOLD) {
            // Noise level exceeds threshold: Turn Buzzer ON (assuming high-active)
```

```
            HAL_GPIO_WritePin(BUZZER_GPIO_PORT, BUZZER_PIN, GPIO_PIN_SET);
        } else {
            // Noise level is acceptable: Turn Buzzer OFF
            HAL_GPIO_WritePin(BUZZER_GPIO_PORT, BUZZER_PIN, GPIO_PIN_RESET);
        }



        // 4. FFT (Spectrum Calculation)         // Simple Hanning window         for (uint32_t i = 0;
i < FFT_SIZE; i++) {             float32_t window = 0.5f * (1.0f - arm_cos_f32(2.0f * 3.1415926535f *
i / (FFT_SIZE - 1)));             fftInput[i * 2] = audioBuffer[i] * window;  // Real part
fftInput[i * 2 + 1] = 0.0f;  // Imag=0
        }


        // Run FFT         arm_rfft_fast_f32(&fftHandler,
fftInput, fftOutput, 0);


        // Magnitudes         float32_t mag[FFT_SIZE / 2];
arm_cmplx_mag_f32(&fftOutput[2], mag, (FFT_SIZE / 2) - 1);  // Start at index 1
        // Anti-flicker: Update only every 4th FFT
static uint32_t updateCounter = 0;
updateCounter++;         if (updateCounter >= 4) {
// ~64ms updates             updateCounter = 0;
// Clear buffer
            SSD1309_Fill(0);
            // 5. DISPLAY OVERALL DB VALUE (using integer-only snprintf workaround)
char db_str[24];


            // Separate integer part
            int32_t intPart = (int32_t)overall_dBFS;
            // Get the first decimal digit             int32_t decPart =
(int32_t) (fabsf(overall_dBFS) * 10.0f) % 10;
            // Format the dB value
            snprintf(db_str, sizeof(db_str), "RMS: %ld.%ld dB (Thr: %.0f)", (long)intPart, (long)decPart,
ALARM_DB_THRESHOLD);
            WriteText(1, 1, (unsigned char*)db_str); // Line 1
            // Spectrum Header
            WriteText(1, 2, (unsigned char*)"SPECTRUM"); // Line 2
```

```
            // 6. DRAW SPECTRUM BARS             uint32_t
spectrum_top_y = 24;             uint32_t max_spectrum_height =
64 - spectrum_top_y;             uint32_t numBars = 64; //
Display 64 bars             for (uint32_t bar = 0; bar <
numBars; bar++) {
                // Group bins for each bar                 uint32_t
totalBins = (FFT_SIZE / 2) - 1;                 uint32_t binStart =
(bar * totalBins) / numBars;                 uint32_t binEnd = ((bar
+ 1) * totalBins) / numBars;
                 float32_t binMag = 0.0f;                 uint32_t numBins = 0;
for (uint32_t b = binStart; b < binEnd && b < totalBins; b++) {
binMag += mag[b];                     numBins++;
                }                 if (numBins > 0) binMag /=
(float32_t)numBins;


                // dBFS for spectrum bars (normalized to FFT_SIZE)
                float32_t dB = (binMag > 0.0001f) ? 20.0f * log10f(binMag * 2.0f / (float32_t)FFT_SIZE) : -
60.0f;                 dB = (dB > 0.0f) ? 0.0f : (dB < -60.0f ? -60.0f
: dB);


                // Height mapping                 uint32_t height = (uint32_t)((dB + 60.0f) *
(float32_t)max_spectrum_height / 60.0f);
                 uint32_t barX = (bar * 128) /
numBars;
                // Draw from Y = (64 - height) to Y = 63 (bottom of screen)
                SSD1309_DrawRectangle(barX, 64 - height, barX + 1, 63, 1);
            }


            // Flush once
            SSD1309_UpdateScreen();
        }
    }


    // Restart DMA reception for the next sample
    HAL_I2S_Receive_DMA(hi2s, &toReceive, 1);
}
/* USER CODE END 0 */
```

```
/**
* @brief  The application entry point.
* @retval int   */ int main(void)
{

  /* USER CODE BEGIN 1 */

  /* USER CODE END 1 */

  /* MCU Configuration--------------------------------------------------------*/

  /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
  HAL_Init();

  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  /* Configure the system clock */
  SystemClock_Config();

  /* USER CODE BEGIN SysInit */

  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
  MX_GPIO_Init();
MX_DMA_Init();
  MX_I2S2_Init();
  MX_SPI1_Init();
  /* USER CODE BEGIN 2 */
  // Set CS high (idle) for OLED SPI
  HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_SET); // Assuming OLED_CS is on PB0

  SSD1309_Reset();
  InitDisplay();
```

```
  ClearScreen();  // Clear for clean start

  // Initial display - simple startup message
  WriteText(1, 1, (unsigned char*)"INIT: dB ALARM");

  // Init CMSIS FFT   arm_status status =
arm_rfft_fast_init_f32(&fftHandler, FFT_SIZE);   if (status !=
ARM_MATH_SUCCESS) {
      Error_Handler();  // Init fail → error
  }

  // Set initial state of buzzer to OFF
  HAL_GPIO_WritePin(BUZZER_GPIO_PORT, BUZZER_PIN, GPIO_PIN_RESET);

  // Start DMA
  HAL_I2S_Receive_DMA(&hi2s2, &toReceive, 1);
  /* USER CODE END 2 */

  /* Infinite loop */   /*
USER CODE BEGIN WHILE */
while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
}
/**
* @brief System Clock Configuration
* @retval None   */ void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

  /** Configure the main internal regulator output voltage
  */
```

```
  __HAL_RCC_PWR_CLK_ENABLE();
  __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);

  /** Initializes the RCC Oscillators according to the specified parameters
* in the RCC_OscInitTypeDef structure.
  */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
  RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
  RCC_OscInitStruct.PLL.PLLM = 8;
  RCC_OscInitStruct.PLL.PLLN = 100;
  RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
RCC_OscInitStruct.PLL.PLLQ = 4;   if
(HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }

  /** Initializes the CPU, AHB and APB buses clocks
  */
  RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
   if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_3) !=
HAL_OK)
  {
    Error_Handler();
  }
}

/**
* @brief I2S2 Initialization Function
* @param None
```

```
* @retval None   */ static void MX_I2S2_Init(void)
{

  /* USER CODE BEGIN I2S2_Init 0 */

  /* USER CODE END I2S2_Init 0 */

  /* USER CODE BEGIN I2S2_Init 1 */

  /* USER CODE END I2S2_Init 1 */   hi2s2.Instance =
SPI2;   hi2s2.Init.Mode = I2S_MODE_MASTER_RX;
hi2s2.Init.Standard = I2S_STANDARD_PHILIPS;
hi2s2.Init.DataFormat = I2S_DATAFORMAT_24B;
hi2s2.Init.MCLKOutput = I2S_MCLKOUTPUT_ENABLE;
hi2s2.Init.AudioFreq = I2S_AUDIOFREQ_16K;
hi2s2.Init.CPOL = I2S_CPOL_LOW;   hi2s2.Init.ClockSource
= I2S_CLOCK_PLL;   hi2s2.Init.FullDuplexMode =
I2S_FULLDUPLEXMODE_DISABLE;   if (HAL_I2S_Init(&hi2s2) !=
HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN I2S2_Init 2 */

  /* USER CODE END I2S2_Init 2 */
}

/**
* @brief SPI1 Initialization Function
* @param None
* @retval None   */ static void MX_SPI1_Init(void)
{

  /* USER CODE BEGIN SPI1_Init 0 */

  /* USER CODE END SPI1_Init 0 */
```

```
  /* USER CODE BEGIN SPI1_Init 1 */

  /* USER CODE END SPI1_Init 1 */   /* SPI1 parameter
configuration*/   hspi1.Instance = SPI1;
hspi1.Init.Mode = SPI_MODE_MASTER;   hspi1.Init.Direction
= SPI_DIRECTION_2LINES;   hspi1.Init.DataSize =
SPI_DATASIZE_8BIT;   hspi1.Init.CLKPolarity =
SPI_POLARITY_LOW;   hspi1.Init.CLKPhase =
SPI_PHASE_1EDGE;   hspi1.Init.NSS = SPI_NSS_SOFT;
hspi1.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_4;
hspi1.Init.FirstBit = SPI_FIRSTBIT_MSB;
hspi1.Init.TIMode = SPI_TIMODE_DISABLE;
hspi1.Init.CRCCalculation = SPI_CRCCALCULATION_DISABLE;
hspi1.Init.CRCPolynomial = 10;   if (HAL_SPI_Init(&hspi1)
!= HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN SPI1_Init 2 */

  /* USER CODE END SPI1_Init 2 */

}

/**
  * Enable DMA controller clock
  */ static void
MX_DMA_Init(void)
{

  /* DMA controller clock enable */
  __HAL_RCC_DMA1_CLK_ENABLE();
__HAL_RCC_DMA2_CLK_ENABLE();

  /* DMA interrupt init */
  /* DMA1_Stream3_IRQn interrupt configuration */
```

```
  HAL_NVIC_SetPriority(DMA1_Stream3_IRQn, 0, 0);
  HAL_NVIC_EnableIRQ(DMA1_Stream3_IRQn);
  /* DMA2_Stream2_IRQn interrupt configuration */
  HAL_NVIC_SetPriority(DMA2_Stream2_IRQn, 0, 0);
  HAL_NVIC_EnableIRQ(DMA2_Stream2_IRQn);

}

/**
* @brief GPIO Initialization Function
* @param None
* @retval None   */ static void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};
  /* USER CODE BEGIN MX_GPIO_Init_1 */
  // Ensure GPIOA clock is enabled
  __HAL_RCC_GPIOA_CLK_ENABLE();
  /* USER CODE END MX_GPIO_Init_1 */

  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOH_CLK_ENABLE();
  __HAL_RCC_GPIOA_CLK_ENABLE();
  __HAL_RCC_GPIOB_CLK_ENABLE();
  /*Configure GPIO pin Output Level */
  HAL_GPIO_WritePin(GPIOB, OLED_RES_Pin|OLED_DC_Pin|OLED_CS_Pin, GPIO_PIN_RESET);

  // Set initial state of Buzzer pin PA8 to OFF
  HAL_GPIO_WritePin(BUZZER_GPIO_PORT, BUZZER_PIN, GPIO_PIN_RESET);

  /*Configure GPIO pins : OLED_RES_Pin OLED_DC_Pin OLED_CS_Pin */
  GPIO_InitStruct.Pin = OLED_RES_Pin|OLED_DC_Pin|OLED_CS_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

  // Configure BUZZER_PIN (PA8)
```

```
  GPIO_InitStruct.Pin = BUZZER_PIN;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(BUZZER_GPIO_PORT, &GPIO_InitStruct);



  /* USER CODE BEGIN MX_GPIO_Init_2 */

  /* USER CODE END MX_GPIO_Init_2 */
}

/* USER CODE BEGIN 4 */

/* USER CODE END 4 */

/**
* @brief  This function is executed in case of error
  occurrence.
* @retval None   */ void Error_Handler(void)
{
  /* USER CODE BEGIN Error_Handler_Debug */
  /* User can add his own implementation to report the HAL error return state */
  __disable_irq();
  while (1)
  {
  }
  /* USER CODE END Error_Handler_Debug */
}
#ifdef USE_FULL_ASSERT
/**
* @brief  Reports the name of the source file and the source line number
* where the assert_param error has occurred.
* @param  file: pointer to the source file name
* @param  line: assert_param error line source number
* @retval None   */ void assert_failed(uint8_t *file, uint32_t line)
{
  /* USER CODE BEGIN 6 */
```

```
  /* User can add his own implementation to report the file name and line number,
ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
  /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */
```